



华东理工大学学报(自然科学版)

Journal of East China University of Science and Technology

ISSN 1006-3080, CN 31-1691/TQ

《华东理工大学学报(自然科学版)》网络首发论文

题目: 基于多通道图神经网络和 CNN-BiLSTM 的漏洞检测方法
作者: 李鹏威, 郑红, 单蓉胜
DOI: 10.14135/j.cnki.1006-3080.20250317001
收稿日期: 2025-03-17
网络首发日期: 2025-07-08
引用格式: 李鹏威, 郑红, 单蓉胜. 基于多通道图神经网络和 CNN-BiLSTM 的漏洞检测方法[J/OL]. 华东理工大学学报(自然科学版).
<https://doi.org/10.14135/j.cnki.1006-3080.20250317001>



网络首发: 在编辑部工作流程中, 稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定, 且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式(包括网络呈现版式)排版后的稿件, 可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定; 学术研究成果具有创新性、科学性和先进性, 符合编辑部对刊文的录用要求, 不存在学术不端行为及其他侵权行为; 稿件内容应基本符合国家有关书刊编辑、出版的技术标准, 正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性, 录用定稿一经发布, 不得修改论文题目、作者、机构名称和学术内容, 只可基于编辑规范进行少量文字的修改。

出版确认: 纸质期刊编辑部通过与《中国学术期刊(光盘版)》电子杂志社有限公司签约, 在《中国学术期刊(网络版)》出版传播平台上创办与纸质期刊内容一致的网络版, 以单篇或整期出版形式, 在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊(网络版)》是国家新闻出版广电总局批准的网络连续型出版物(ISSN 2096-4188, CN 11-6037/Z), 所以签约期刊的网络版上网络首发论文视为正式出版。

DOI: 10.14135/j.cnki.1006-3080.20250317001

基于多通道图神经网络和 CNN-BiLSTM 的漏洞检测方法

李鹏威¹, 郑红¹, 单蓉胜²

(1. 华东理工大学信息科学与工程学院, 上海 200237; 2. 上海交通大学网络空间安全学院, 上海 200240)

摘要:针对当前基于深度学习的漏洞检测方法大多仅考虑代码序列语义或代码结构语义的问题, 本文提出了基于多通道图神经网络(Multi Channel Graph Neural Network)和 CNN-BiLSTM 的漏洞检测方法 MCGCBVul。该方法通过中心性分析将代码属性图的节点特征矩阵扩展为多通道的类图像矩阵, 并使用图注意力网络(GATv2)和二维卷积神经网络(2D-CNN)提取图结构特征。同时采用双尺度一维卷积神经网络(1D-CNN)以及双向长短期记忆网络(BiLSTM)提取序列特征。最终将图特征和序列特征进行特征融合, 以达到更好的漏洞检测性能。在 FFMpeg+Qemu 和 Reveal 两个数据集上进行实验后, 实验结果表明, MCGCBVul 在 F1 得分、准确率等多项指标上优于文中 6 种对比模型, 准确率分别达到了 63.952% 和 92.007%。此外, 本文通过消融实验进一步证明了模型各模块改进的有效性。

关键词:漏洞检测; 图神经网络; 特征融合; 中心性分析; 深度学习

中图分类号: TP311

文献标志码: A

随着互联网技术的不断发展, 各类计算机软件已成为人们生活中不可或缺的一部分, 为人们的生活带来了极大的便利。然而伴随计算机产业规模的快速增长, 软件的组成与功能也日渐复杂, 从而导致软件漏洞的数量也随之攀升。软件漏洞的出现可能会导致软件程序崩溃、用户隐私泄露等问题, 进而造成软件开发者的经济损失并给用户的使用带来安全风险^[1]。而由于软件开发者编写代码时的疏忽以及编程语言本身可能存在的局限性, 大部分软件都难以避免地会引入漏洞。因此, 研究软件漏洞检测方法, 提高软件系统的安全性和稳定性, 具有重要的现实意义和实际价值。

传统的漏洞检测方法通常依赖于人工定义的漏洞规则。但这种人工规则的好坏取决于专家的知识水平和经验, 不完善的规则往往会导致漏洞的误报和漏报, 而定义较为完善的规则又需要很高的人力成本, 因此这种方法漏洞检测效果不够理想^[2]。如今, 深度学习技术的快速发展为代码漏洞检测方法提供了新的思路。基于深度学习的漏洞检测方法一

般从源代码中提取特征生成嵌入, 然后输入到各类神经网络模型中进行进一步特征提取, 并最终生成二分类结果。这种方法可以自动提取代码特征, 避免了人工定义漏洞规则的成本问题, 也提升了漏洞检测效果。早期的研究方法主要为基于序列的方法, 即将代码视为文本序列, 并采用自然语言序列的相关模型提取序列特征进行漏洞检测, 但这种方法忽略了代码包含的深层次结构语义信息。基于图的漏洞检测方法则根据源代码构建抽象语法树、程序依赖图等代码图表示, 并利用适用于图结构的图神经网络模型有效地提取了代码结构语义信息。但另一方面, 基于图的方法则在提取序列语义上存在不足。如今, 深度学习漏洞检测的重点主要在于研究如何更好地抽取和利用源代码中的各类语义信息获取更好的漏洞表征以及研究更高效的深度学习模型来提升漏洞检测效果。

针对上述问题, 本文提出了基于多通道图神经网络(Multi Channel Graph Neural Network)和 CNN-BiLSTM 的漏洞检测模型 MCGCBVul。该模型主要

收稿日期: 2025-03-17

作者简介: 李鹏威(2001—), 男, 江苏人, 硕士生, 主要研究方向: 代码漏洞检测。E-mail: y80220073@mail.ecust.edu.cn

通信联系人: 郑红, E-mail: zhenghong@ecust.edu.cn

分为两个模块,即图特征提取模块和序列特征提取模块。这两个模块分别提取代码图特征和序列特征并在融合后进行漏洞检测。其中图特征即代码图表示中所包含的结构语义特征。图特征提取模块引入了中心性分析将 CPG 特征矩阵扩展为了包含多种语义的多通道特征矩阵,并采用共享权重的多通道图神经网络和 2D-CNN 提取其中的代码结构语义特征。序列特征提取模块通过双尺度 CNN、BiLSTM 以及注意力机制提取源代码中的序列语义。最后将 MCGCBVul 两种语义相融合进行漏洞检测并实现了相较于对比模型更优的漏洞检测性能。综上所述,本文的主要贡献如下:

本文通过引入中心性分析增强了代码语义,并采用多通道图神经网络和 2D-CNN 模型有效捕捉了代码 CPG 中的结构语义信息;利用双尺度一维卷积神经网络(1D-CNN)、BiLSTM 以及注意力机制构建了序列特征提取模块,有效地提取了源代码的序列语义信息;提出了 MCGCBVul 模型,该模型可以有效地利用图神经网络和 CNN-BiLSTM 模型提取代码的图特征以及序列特征并融合,从而达到较好的漏洞检测性能;在两个真实数据集上进行实验,并证明了 MCGCBVul 的优越性。

1 相关工作

目前基于深度学习的漏洞检测方法,主要分为基于序列和基于图两种方法。

基于序列的漏洞检测方法将源代码视为自然语言序列,并采用自然语言处理相关的深度学习模型来检测漏洞。目前,诸如循环神经网络(RNN)^[3]、卷积神经网络(CNN)以及长短期记忆网络(LSTM)^[4]等经典的深度学习模型已经应用于漏洞检测。Li 等^[5]提出的漏洞检测方法 VulDeePecker 将切片后的代码视为纯文本,然后输入双向长短期记忆网络(BiLSTM)中提取序列特征,实现了切片级的漏洞检测。之后, Li 等^[6]又提出了 SySeVR,该方法采用 BGRU 模型进行漏洞检测,并取得了比 VulDeePecker 更好的漏洞检测效果。由于 BERT^[7]、RoBERTa^[8]等预训练语言模型在自然语言处理应用中取得了重大成功,而代码本身也属于语言序列,因此研究人员也开始将预训练语言模型应用到代码任务中。Feng 等^[9]提出了用于自然语言和编程语言的双模态预训练模型 CodeBERT。CodeBERT 学习自然语言与代码的通用表示,可以用于自然语言代码搜索、代码文档生成等下游任务。Lu 等^[10]将 CodeBERT 应用到代码漏

洞检测任务上,并取得了相较 BiLSTM、CNN 等模型更好的漏洞检测准确率。

然而,基于序列的漏洞检测方法仅仅把代码视为自然语言序列,并没有考虑到隐藏在代码中的深层次结构信息。为了更好地提取代码漏洞特征,研究人员开始尝试基于源代码绘制各类代码图表示,如抽象语法树(Abstract Syntax Tree, AST)、数据流图(Data Flow Graph, DFG)、控制流图(Control Flow Graph, CFG)、程序依赖图(Program Dependence Graph, PDG)以及代码属性图(Code Property Graph, CPG)^[11]等。而为了匹配图这一数据结构,研究人员则采用图神经网络(Graph Neural Networks, GNN)提取图中的结构语义,从而进行漏洞检测。

Zhou 等^[12]提出的 Design 首次将图神经网络应用到漏洞检测任务。该方法首先构建了融合 AST、CFG、DFG 和自然代码序列(Natural Code Sequence, NCS)的代码异构图,然后采用了门控图神经网络(Gated Graph Neural Network, GGNN)^[13]学习节点特征,最后通过一维卷积神经网络(1D-CNN)提取与图级任务相关的特征,从而实现漏洞特征的提取。BGNN4VD^[14]将 AST、CFG 和 DFG 相结合提出了代码合成图,并构建了双向图神经网络(Bidirectional Graph Neural Network, BGNN)学习漏洞特征。Saikat 等^[15]提出的 ReVeal 使用门控图神经网络在代码属性图上学习代码图结构特征,从而进行漏洞检测。ReGVD^[16]使用滑动窗口的方式将原始代码序列构建合成图,然后采用残差 GCN 提取代码图的特征,并利用图级池化层输出图向量进行漏洞检测,取得了相较于 CodeBERT 和 GraphCodeBert^[17]更好的检测性能。Xiao 等^[18]提出的 EnGS2F 将 PDG 和上下文关系图(Context Relationship Graph, CRG)相组合构建了切片关系图(Slicing Relation Graph, SRG),并将门控图神经网络和图注意力机制相融合以提升模型的漏洞特征学习能力。LineVD^[19]使用 CodeBert 提取源代码函数级和语句级的特征,并使用图注意力网络(Graph Attention Network, GAT)^[20]对图中的节点进行分类,从而实现了语句级的漏洞检测任务。VulCNN^[21]使用 Joern 将代码转换为程序依赖图,并采用将节点中心性将程序依赖图转换为三通道的图像,然后使用 CNN 进行漏洞检测,取得了较好的准确性和可扩展性。

2 MCGCBVul 模型

本节主要介绍漏洞检测模型的实现流程。

MCGCBVul 模型的整体架构如图 1 所示。

MCGCBVul 同时提取代码图结构语义特征和序列语义特征, 并将两种特征融合后实现漏洞检测。首先, MCGCBVul 基于 Joern 将代码转化为代码属性图 (Code Property Graph, CPG), 并采用预训练模型 GraphCodeBert 模型将代码节点进行向量化表征, 以全面高效地表示 CPG 中所包含的语法语义等结构信息。之后, 引入中心性分析来度量 CPG 节点的重要性, 并通过不同中心性构建多通道特征矩阵, 然后采用共享权重的多通道图神经网络 GATv2 和二维卷积神经网络 (2D-CNN) 提取多通道 CPG 中的图特征并

对其进行有效融合。同时, MCGCBVul 采用预训练模型 CodeBert 对源代码进行分词嵌入, 并构建了 CNN-BiLSTM 模型提取源代码中的序列特征。CNN-BiLSTM 模型采用两种不同卷积核大小的 1D-CNN, 分别提取了代码序列不同尺度下的语义特征。然后 CNN-BiLSTM 模型采用 BiLSTM 捕捉代码序列的全局特征, 并用注意力机制增强关键 token 的重要性, 从而进一步提升了模型的特征提取能力。最终 MCGCBVul 将图特征和序列特征进行拼接并输入全连接层进行二分类任务, 从而实现代码漏洞检测。

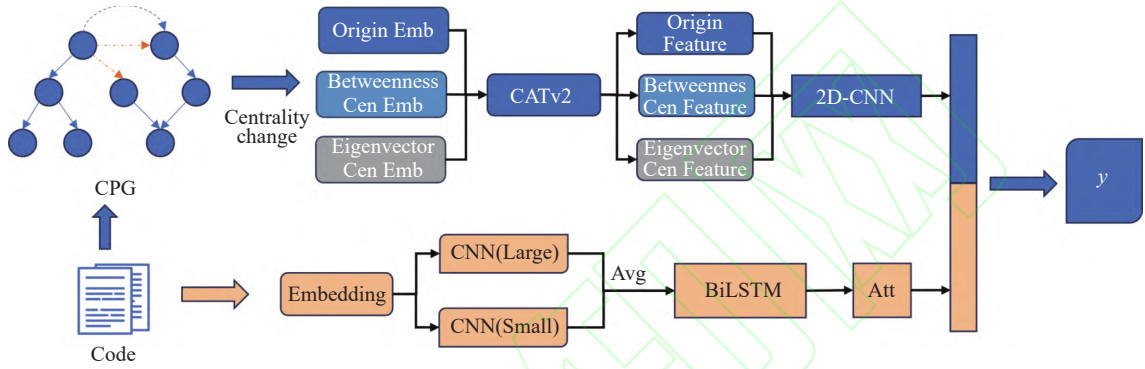


图 1 MCGCBVul 模型架构

Fig. 1 Architecture of the MCGCBVul

2.1 图特征提取模块

2.1.1 图构建 代码作为一种编程语言仍属于自然语言序列, 而代码序列本身并不能体现其包含的复杂语法语义等结构信息。因此需要将代码转换为各类代码图表示, 以图的形式将代码包含的结构信息显式表达出来。不同的代码图表示包含不同的结构语义。在过去基于图的漏洞检测方法中, 研究人员选择了抽象语法树 (AST)、控制依赖图 (PDG)、控制流图 (CFG) 以及代码属性图 (CPG) 等等代码图表示作为图神经网络的输入。而在众多代码图表示中, CPG 将 AST、PDG 和 CFG 这 3 种表示方式整合到一个基于 AST 的统一数据结构中, 结合了 AST、CFG 和 PDG 各自的特点, 包含了代码语法、数据流以及控制流等多种语义信息, 更为充分地表达了代码图结构语义。因此本文选择 CPG 这一更高效的图表示方法作为图结构。本文使用开源软件 Joern 将原始代码转化为 CPG。

2.1.2 图节点向量化 CPG 中每一个节点中包含了某个代码块的具体代码以及其节点类型。本文将节点类型和节点代码进行拼接作为最终的节点代码, 以获取节点类型语义。为了输入到后续的图神经网络训练, 需要将节点代码转化为固定长度的向量。

本文采用 GraphCodeBert 将节点代码进行向量化表征。GraphCodeBert 是一个基于 Transformer 的预训练模型, 它通过在预训练阶段引入数据流从而有效提取了代码结构语义信息, 因此具有较好的代码表征效果。具体而言, 本文通过 GraphCodeBert 的 tokenizer 模块将节点代码进行分词生成多个 token, 然后对每个 token 向量化后取其平均得到最终的节点嵌入。

2.1.3 多通道特征生成 在完成 CPG 中每个节点的嵌入后, 为了进一步丰富节点语义特征, 本文引入网络中心性分析的方法来获得所有节点在 CPG 的重要性。为了从多角度考虑 CPG 节点的特征, 本文将采用中介中心性和特征向量中心性这两种中心性将原始节点特征矩阵转为两种代表不同中心性特点的特征矩阵。这两类中心性的详细说明如下。

(1) 中介中心性

中介中心性通过计算一个节点出现在任意两个节点之间最短路径的次数来衡量节点的重要性。

$$X_i = \sum_{j,k \in N} \frac{B(i,j,k)}{B(j,k)} \quad (1)$$

X_i 是 i 节点的中介中心性值, N 是图中总节点数, j, k 代表任意两个节点。 $B(j, k)$ 为 j, k 两点之间

最短路径的总数。 $B(i,j,k)$ 为*i*节点出现在*j,k*两点之间最短路径的次数。

(2) 特征向量中心性

特征向量中心性通过综合与其相连的节点的中心性来计算当前节点的中心性。

$$\mathbf{X}_i = \alpha \sum_j A_{ij} \mathbf{x}_j + \beta \quad (2)$$

其中, $\mathbf{X}_i$ 是*i*节点的特征向量中心性值, $\mathbf{x}_j$ 是*j*节点的特征向量中心性值。

A_{ij} 为邻接矩阵, *i* 代表当前节点, *j* 为与当前节点直连的节点。参数 β 控制初始中心性, 参数 α 需要满足:

$$\alpha < \frac{1}{\lambda_{\max}} \quad (3)$$

其中 λ 为邻接矩阵 A_{ij} 的特征值。

在得到每个节点的中心性后, 本文通过节点嵌入和其对应中心性直接相乘的方式, 得到新的节点嵌入。具体方法如图2所示。

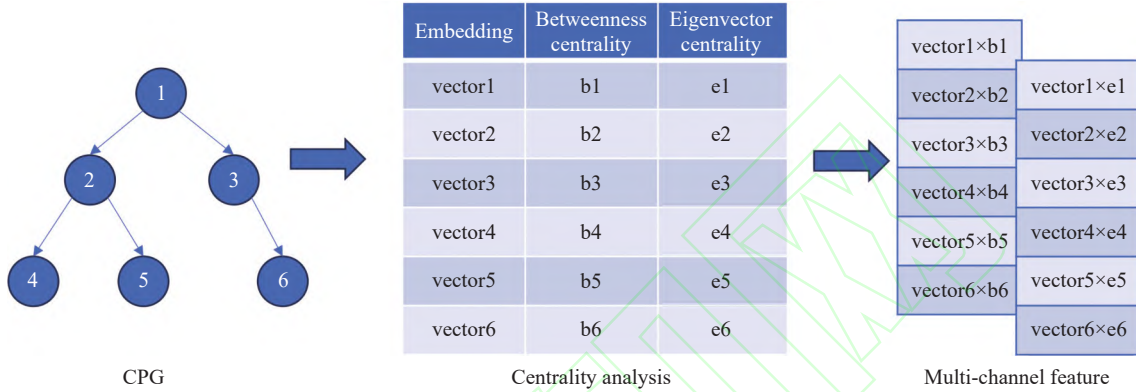


图2 中心性变化图

Fig. 2 Centrality Change Graph

由于采用了两种中心性, 因此通过中心性变换可以得到两种代表不同中心性的特征矩阵。但将节点嵌入与中心性相乘可能会导致丢失掉原本嵌入的某些信息, 因此本文仍然保留原始的嵌入, 并将其和中心性生成的特征矩阵结合成一个三通道特征矩阵。

2.1.4 图特征提取模型 为了更好地提取三通道特征矩阵中的图特征, 本文采用图注意力网络(GATv2)构建了多通道图神经网络。GATv2^[22]是GAT的改进版本, 其通过引入动态注意力机制解决了GAT中存在的注意力系数和查询节点无关的缺点, 提升了模型表达能力。本文采用并行的方式, 针对每个通道的特征矩阵均用GATv2学习特征表示。具体来说, 该网络通过一个共享权重的GATv2学习3个不同通道的图特征, 从而构建多通道图神经网络, 获取多维图特征。GATv2的公式如式(4):

$$h'_i = \sigma \left(\sum_{j \in N(i)} \alpha_{i,j} W h_j \right) \quad (4)$$

其中, h'_i 为*i*节点新一层的特征, h_j 为*j*节点特征, W 为权重矩阵, σ 为激活函数, $N(i)$ 表示所有节点的集合。 $\alpha_{i,j}$ 为节点*i*和*j*的注意力系数, 具体公式如式(5):

$$\alpha_{i,j} = \frac{\exp(a^T \text{LeakyReLU}(W \cdot [h_i || h_j]))}{\sum_{k \in N(i)} \exp(a^T \text{LeakyReLU}(W \cdot [h_i || h_k]))} \quad (5)$$

其中, $W \in \mathbb{R}^{d \times d}$ 和 $a \in \mathbb{R}^{2d}$ 为可学习参数, LeakyReLU 激活函数, $||$ 为拼接操作。

在通过多通道 GATv2 分别学习包含不同中心性特点的图特征后, 本文采用 CNN 对三通道 GATv2 输出的特征矩阵进行进一步的特征提取。由于需要处理三个通道的输入, 因此本文采用二维卷积 Conv2d 对三个通道的特征矩阵进行卷积。同时本文采用了多个大小不同的卷积核来捕获特征矩阵不同维度的图特征。每种大小的卷积核各有 32 个。较小的卷积核可以学习到节点之间的近距离依赖关系, 而较大的卷积核则可以学习到节点之间的远距离依赖关系。最终将所有卷积核的输出汇聚为一个一维张量, 作为代码的图特征。具体公式如式(6):

$$\mathbf{H}'_g = \text{FC} \left(\left\| \bigoplus_{k=1}^K \text{MaxPool}(\sigma(\text{Conv2D}_k(\mathbf{H}_g))) \right\| \right) \quad (6)$$

其中 $\mathbf{H}_g$ 为卷积层的输入, 即三通道的特征矩阵, $\mathbf{H}'_g$ 为聚集多通道输入后的特征向量。 k 为卷积核的大小, K 为卷积核的总数。 σ 为激活函数, MaxPool 为最大池化操作, FC 为全连接层。

2.2 序列特征提取模块

序列特征提取模块将函数代码视为自然语言序

列以提取源代码本身的序列语义。为了保留代码作为序列的原始语义, 本文直接采用原始代码作为序列模型的输入。同时为了更好地提取代码序列语义, 本文采用 CodeBert 对源代码进行嵌入, 得到词嵌入矩阵。然后, 本文采用 CNN-BiLSTM 模型来进一步提取序列语义, 该模型由两个不同大小卷积核的一维卷积神经网络(1D-CNN), 双向长短期记忆网络(BiLSTM)以及注意力机制组成。具体来说, 首先将代码特征矩阵采用并行的方式分别输入到两个不同大小卷积核的 1D-CNN 中。其中较小卷积核的 1D-CNN 可以提取代码序列的短距离依赖关系, 而较大卷积核的 1D-CNN 则可以捕捉较长距离的依赖关系, 有助于识别依赖较远上下文逻辑的潜在漏洞模式。然后将两者得到的特征进行平均, 使得模型能够同时考虑短距离和较长距离的依赖关系, 捕捉代码序列不同尺度的序列特征。公式如式 (7):

$$H_c = \text{AVG}(\text{Conv1D}_1(H), \text{Conv1D}_2(H)) \quad (7)$$

其中 H 为词嵌入矩阵, Conv1D_1 代表小卷积核的 CNN, Conv1D_2 代表大卷积核的 CNN, AVG 为平均操作。

接着, 为了从整个代码函数的维度更全面地捕捉漏洞模式, 本文引入双向长短期记忆网络(BiLSTM)来提取源代码的全局特征。相较于 CNN, BiLSTM 通过记忆机制保留历史信息, 可以更好地建模长距离依赖关系。在程序代码中, 代码行的一些变量既有可能受到早期语句的影响, 同时也可能受到后期语句的影响, 而许多漏洞的产生也通常涉及前后代码的逻辑关系, 因此通过 BiLSTM 建模代码序列的双向语义, 可以充分提取代码序列的上下文特征, 从而更全面地提取代码序列中所包含的深层次漏洞特征。具体方法如式 (8) 所示。

$$H_b = \text{BiLSTM}(H_c) \quad (8)$$

其中, H_c 为 CNN 处理后的特征矩阵。

最后利用注意力机制进一步动态分配不同时间步的权重, 增强对重要特征的关注, 使得模型在学习过程中关注到更可能出问题的代码片段从而提高模型的可解释性, 增强捕获漏洞特征的能力。将注意力权重与特征矩阵相乘后得到最终的序列特征。

$$H_s = H_b * \text{Softmax}(\tanh(H_c)W) \quad (9)$$

其中, H_b 为 BiLSTM 处理后的特征矩阵, W 为注意力权重参数。

2.3 特征融合

在经过图特征提取模块和序列特征提取模块的

特征提取后, 本文采用拼接操作对两种特征进行融合, 然后输入全连接层进行二分类, 实现漏洞检测。具体公式如式 (10):

$$y = \text{FC}(H_g \parallel H_s) \quad (10)$$

其中, H_g 为图特征向量, H_s 为序列特征向量, $\parallel$ 为拼接操作, FC 为全连接层, y 为二分类输出。

3 实验与分析

3.1 数据集

本文采用了 FFMpeg+Qemu^[12]、Reveal^[15] 两个漏洞数据集进行实验, 用以评估 MCGCBVul 模型的有效性。FFMpeg+Qemu 数据集来自于两个大型的 C 语言开源项目 FFMpeg 和 Qemu, 其包含了大约 11k 个漏洞函数和 14k 个非漏洞函数, 为平衡数据集。Reveal 数据集则是一个不平衡数据集, 总共包含了大约 19k 个函数, 其中仅有 2000 个函数为漏洞函数。

表 1 数据集信息统计

Dataset	Samples	Vul	Non-vul
FFMpeg+Qemu	25,615	11,673	13,942
Reveal	19,593	2,000	17,593

3.2 实验设置

在图特征的提取上, 本文采用 2D-CNN 模型对图神经网络输出的多通道特征进行卷积。对于 2D-CNN 的卷积核设置, 首先设 2D-CNN 卷积核的形状为 $m*n$ 。其中 m 为卷积时同时考虑的节点数。 n 为张量维度, 本文中 n 的大小设置为 128。为了同时捕捉代码中远程依赖关系和近距离依赖关系, m 的取值由小到大被设置为了 1, 2, 3, 4, 5, 10, 16, 17, 18, 19, 20。每种不同大小的卷积核各有 32 个。

在序列特征的提取上, 本文采用 CNN-BiLSTM 模型, 其中 1D-CNN 设置了一大一小两个卷积核以提取较长距离和短距离的依赖关系, 其中大卷积核大小为 15, 小卷积核大小为 3。

在数据集的设置上, 将两个数据集随机划分为训练集、验证集和测试集, 比例为 8 : 1 : 1。对于 MCGCBVul 的实验将 dropout 设置为 0.5, 学习率设置为 0.0001, batch size 设置为 32, 选择 Cross Entropy 作为损失函数, Adam 作为优化器, 最多迭代次数为 20。本文采用搭载了 NVIDIA GeForce RTX 4090 GPU 的服务器进行实验。

3.3 评估标准

本文使用以下 4 个性能指标来评估模型的漏洞检测效果。

(1) 精确率(Precision): 模型预测为正例样本中实际为正例样本的比例。TP 为将正例样本预测为正例的数量, FP 为将负例样本预测为正例的数量。

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (11)$$

(2) 召回率(Recall): 模型正确预测为正例样本的比例。FN 为将正例样本预测为负例的数量。

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (12)$$

(3) F1 得分(F1 score): 精确率和召回率的调和

平均。

$$\text{F1score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (13)$$

(4) 准确率(Accuracy): 预测正确的样本与所有样本的比例。TN 为将负例样本预测为负例的数量。

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FN} + \text{FP}} \quad (14)$$

3.4 结果分析

3.4.1 对比实验 为了评估 MCGCBVul 模型的性能, 本文将 MCGCBVul 与几种先进的基于深度学习的漏洞检测方法进行比较, 包括 Devign、VulCNN、W. Cai et al.、CodeBert、GraphCodeBert 以及 ReGVD。实现结果见表 2 和表 3。

表 2 FFMpeg+Qemu 数据集实验结果对比

Table 2 Comparison of Experimental Results on FFMpeg+Qemu Dataset

Method	Precision	Recall	F1 score	Accuracy
CodeBert ^[9]	60.994	47.059	53.128	61.910
GraphCodeBert ^[17]	61.927	46.036	52.812	62.260
VulCNN ^[21]	53.271	34.015	41.519	56.042
W. Cai et al. ^[23]	53.627	38.188	44.609	56.403
Devign(ReGVD) ^[12]	53.825	49.190	51.403	57.330
ReGVD ^[16]	61.576	43.308	50.851	61.600
Ours	63.400	50.831	56.389	63.952

表 3 Reveal 数据集实验结果对比

Table 3 Comparison of Experimental Results on Reveal Dataset

Method	Precision	Recall	F1 score	Accuracy
CodeBert ^[9]	62.162	34.500	44.373	91.170
GraphCodeBert ^[17]	63.462	33.000	43.421	91.220
VulCNN ^[21]	33.333	1.500	2.871	89.616
W. Cai et al. ^[23]	37.037	4.545	8.097	89.916
Devign(ReGVD) ^[12]	73.684	7.000	12.785	90.260
ReGVD ^[16]	57.647	24.500	34.386	90.460
Ours	72.562	34.833	47.065	92.007

实验结果表明, 在 FFMpeg+Qemu 数据集上, 本章提出的 MCGCBVul 模型在精确率、召回率、F1 分数以及准确率上取得了最优结果, 分别达到了 63.400%, 50.831%, 56.389% 和 63.952%。在 Reveal 数据集上, MCGCBVul 在召回率、F1 分数以及准确率上也分别达到了 34.833%, 47.065%, 46.64% 和

92.007%, 高于其他对比模型。这也充分说明了 MCGCBVul 模型相对比较模型的优越性。

在对比模型中, CodeBert 和 GraphCodeBert 为基于代码序列的方法。基于 CodeBert 的方法即在漏洞数据集上对 CodeBert 模型进行微调。由于 CodeBert 已基于大量的代码进行了预训练, 因此其表现出了

良好的漏洞检测效果。但该方法仅仅考虑了代码序列语义, 缺乏了代码结构语义, 因此其在精确率、召回率、F1 得分和准确率上均低于本文的 MCGCBVul 模型。对于 FFMpeg+Qemu 数据集, GraphCodeBert 在准确率和精确率上相较 CodeBert 分别提升了 0.35% 和 0.933%, 但在召回率和 F1 分数上则分别降低了 1.023% 和 0.316%。GraphCodeBert 在预训练阶段引入数据流信息, 并从源代码和数据流两个维度学习代码特征。但其仅考虑了数据流这一种结构语义, 因此在漏洞检测任务上的性能与 CodeBert 大致相同。不同的是 GraphCodeBert 更侧重于准确率和精确率, 而 CodeBert 则侧重于召回率和 F1 分数。VulCNN 构建了代码的程序依赖图, 采用度中心性、接近中心性以及 Katz 中心性分析将 PDG 扩展为三通道的图像, 并利用 2D-CNN 进行漏洞检测。该方法有效利用了 PDG 的控制流和数据流语义, 但并未考虑代码序列语义, 且仅采用了 2D-CNN 模型提取特征, 因此在检测性能上相对较低。W. Cai et al. 为本文对 Cai 等^[23]论文在最大程度上进行复现的方法, 由于其未提供具体代码保留字, 因此在复现时忽略了这一点。W. Cai et al. 将 CPG 拆分为 AST、CFG、PDG, 然后对这个三个子图分别进行中心性分析。其采用了度中心性、接近中心性以及特征向量中心性共三种中心性将三个子图拓展为了一个九通道的图像, 然后采用 2D-CNN 实现通道间的特征融合。通过从更多维度衡量代码节点的重要性, W. Cai et al. 在两个数据集的各项指标上均优于 VulCNN。但 W. Cai et al. 仍然欠缺序列语义, 且 2D-CNN 对图结构语义的提取不够充分, 因此在性能上低于本文模型。Devign 由于未提供代码, 因此本文采用 ReGVD 提供的方案来进行复现。即在代码图表示上采用 ReGVD 的绘图方案, 模型上则采用 Devign 自身的 GGNN 和 CNN 来提取漏洞特征。相对于 VulCNN, GGNN 和 CNN 的结合能够更好地提取代码中包含的漏洞特征, 也因此取得了相对更好的检测效果。ReGVD 用滑动窗口的方式将原始代码序列构建成图, 然后采用残差 GCN 提取代码图的特征, 但其在本质上仍属于在代码序列层面上提取特征, 而缺乏代码结构语义。这些方法也证明了代码序列语义和代码结构语义对漏洞检测任务中都有重要作用。

综上所述, 本文提出的 MCGCBVul 模型采用共享权重的多通道图神经网络和 CNN-BiLSTM 充分提取了代码结构语义和序列语义, 并对两种语义进行了有效融合, 因此在代码漏洞检测的综合性能上高于上述比较模型。

3.4.2 消融实验 本文将完整的 MCGCBVul 模型在 FFMpeg+Qemu 数据集上与分别去除了图特征提取模块和序列特征提取模块后的模型变体进行比较, 以评估不同模块对漏洞检测性能的贡献程度。仅采用图特征提取模块的变体为 MCGCBVul(graph), 仅采用序列特征提取模块的变体为 MCGCBVul(seq)。

如表 4 所示, 完整模型在各项指标上均高于仅使用单一模块的方法, 这充分说明了综合图结构特征和序列特征能够有效提升代码漏洞检测效果。

表 4 不同模块对模型效果的影响

Method	Precision	Recall	F1 score	Accuracy
MCGCBVul(graph)	57.500	27.451	37.161	57.411
MCGCBVul(seq)	62.231	41.858	50.051	61.674
MCGCBVul	63.40	50.831	56.389	63.952

由于在图特征提取模块中, 本文引入了中心性算法来衡量不同节点的重要程度, 并以此为基础构建了多通道特征, 因此为了评估中心性分析的有效性, 本文将完整模型与去除中心性的模型变体进行比较。其中, 去除中心性的模型变体 MCGCBVul(w/o cen) 即是仅通过原始嵌入构建三通道特征。实验结果见表 5。

表 5 中心性对模型效果的影响

Method	Precision	Recall	F1 score	Accuracy
MCGCBVul(w/o cen)	63.315	48.465	54.867	63.463
MCGCBVul	63.40	50.831	56.389	63.952

实验表明完整模型相对去除中心性的模型变体在各项指标上都有相应的提升。因此通过引入中心性算法衡量节点的重要性, 可以从更多维度丰富节点代码语义, 使得模型能够捕捉更多代码漏洞模式, 提升漏洞检测性能。

4 结束语

本文提出了一个基于多通道图神经网络和 CNN-BiLSTM 的漏洞检测模型 MCGCBVul。该模型一方面通过引入中心性算法将原始的 CPG 节点特征矩阵扩展成多通道特征矩阵, 并利用多通道图神经网络和 2D-CNN 来提取图特征, 另一方面采用 CNN-BiLSTM 模型提取代码序列特征, 最终将两种特征融

合后进行漏洞检测。该方法充分提取并融合了代码结构语义和序列语义,实验表明其相较于文中6种比较模型具有更好的代码漏洞检测性能。

在未来的研究中,将继续探索更有效的图结构,通过引入更多类型的边增强代码结构语义。此外,当前工作的特征融合方法仅采用拼接这一简单操作,后续将会进一步研究更高效的特征融合方法以进一步提升漏洞检测性能。

参考文献:

- [1] 詹奇, 潘圣益, 胡星, 等. 开源软件漏洞感知技术综述 [J]. 软件学报, 2024, 35(1): 19-37.
- [2] 苏小红, 郑伟宁, 蒋远, 等. 基于学习的源代码漏洞检测研究与进展 [J]. 计算机学报, 2024, 47(2): 337-374.
- [3] SCHUSTER M, PALIWAL K K. Bidirectional recurrent neural networks[J]. *IEEE Transactions on Signal Processing*, 1997, 45(11): 2673-2681.
- [4] HOCHREITER S, SCHMIDHUBER J. Long short-term memory[J]. *Neural computation*, 1997, 9(8): 1735-1780.
- [5] LI Z, ZOU D, XU S, *et al.* VulDeePecker: A deep learning-based system for vulnerability detection[C]//Proceedings 2018 Network and Distributed System Security Symposium. San Diego (USA): Internet Society, 2018.
- [6] LI Z, ZOU D, XU S, *et al.* Sysevr: A framework for using deep learning to detect software vulnerabilities[J]. *IEEE Transactions on Dependable and Secure Computing*, 2021, 19(4): 2244-2258.
- [7] DEVLIN J, CHANG M W, LEE K, *et al.* Bert: Pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers). 2019: 4171-4186.
- [8] LIU Y, OTT M, GOYAL N, *et al.* Roberta: A robustly optimized bert pretraining approach[J]. *arXiv preprint arXiv: 1907.11692*, 2019.
- [9] FENG Z, GUO D, TANG D, *et al.* CodeBERT: A Pre-Trained Model for Programming and Natural Languages[C]//Findings of the Association for Computational Linguistics: EMNLP 2020. Online: Association for Computational Linguistics, 2020: 1536-1547.
- [10] LU S, GUO D, REN S, *et al.* CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation[C]//Proceedings of the Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track. Online: NeurIPS, 2021.
- [11] YAMAGUCHI F, GOLDE N, ARP D, *et al.* Modeling and discovering vulnerabilities with code property graphs[C]//Proceedings of the 35th IEEE Symposium on Security and Privacy. San Jose (USA): IEEE Computer Society, 2014: 590-604.
- [12] ZHOU Y, LIU S, SIOW J, *et al.* Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks[J]. *Advances in neural information processing systems*, 2019, 32.
- [13] LI Y, ZEMEL R, BROCKSCHMIDT M, *et al.* Gated graph sequence neural networks[C]//Proceedings of the 4th International Conference on Learning Representations. San Juan, Puerto Rico (USA): International Conference on Learning Representations, 2016.
- [14] CAO S, SUN X, BO L, *et al.* Bgcn4vd: Constructing bidirectional graph neural-network for vulnerability detection[J]. *Information and Software Technology*, 2021, 136: 106576.
- [15] CHAKRABORTY S, KRISHNA R, DING Y, *et al.* Deep learning based vulnerability detection: Are we there yet?[J]. *IEEE Transactions on Software Engineering*, 2021, 48(9): 3280-3296.
- [16] NGUYEN V A, NGUYEN D Q, NGUYEN V, *et al.* Regvd: Revisiting graph neural networks for vulnerability detection[C]//Proceedings of the 44th ACM/IEEE International Conference on Software Engineering: Companion Proceedings. Pittsburgh (USA): Association for Computing Machinery (ACM), 2022: 178-182.
- [17] GUO D, REN S, LU S, *et al.* GraphCodeBERT: Pre-training Code Representations with Data Flow[C]//Proceedings of the 9th International Conference on Learning Representations. Online: International Conference on Learning Representations, 2021.
- [18] XIAO P, XIAO Q, ZHANG X, *et al.* Vulnerability detection based on enhanced graph representation learning[J]. *IEEE Transactions on Information Forensics and Security*, 2024.
- [19] HIN D, KAN A, CHEN H, *et al.* LineVD: Statement-level vulnerability detection using graph neural networks[C]//Proceedings of the 19th International Conference on Mining Software Repositories. Pittsburgh (USA): IEEE/ACM, 2022: 596-607.
- [20] VELIČKOVIĆ P, CUCURULL G, CASANOVA A, *et al.* Graph Attention Networks[C]//Proceedings of the 6th International Conference on Learning Representations. Vancouver (Canada): International Conference on Learning Representations, 2018.
- [21] WU Y, ZOU D, DOU S, *et al.* Vulcnn: An image-inspired scalable vulnerability detection system[C]//Proceedings of the 44th International Conference on Software Engineering. Pittsburgh (USA): IEEE/ACM, 2022: 2365-2376.
- [22] BRODY S, ALON U, YAHAV E. How Attentive are

Graph Attention Networks?[C]//Proceedings of the 10th International Conference on Learning Representations. Online: International Conference on Learning Representations, 2022.

[23] CAI W, CHEN J, YU J, *et al.* A software vulnerability detection method based on deep learning with complex network analysis and subgraph partition[J]. [Information and Software Technology](#), 2023, 164: 107328.

Vulnerability detection method based on multi channel graph neural network and CNN-BiLSTM

LI Pengwei¹, ZHENG Hong¹, SHAN Rongsheng²

(1. School of Information Science and Engineering, East China University of Science and Technology; 2. School of Cyber Science and Engineering, Shanghai Jiao Tong University)

Abstract: In view of the problem that most of the current vulnerability detection methods based on deep learning only consider the semantics of code sequence or code structure, this paper proposes a vulnerability detection method MCGCBVul based on Multi Channel Graph Neural Network and CNN-BiLSTM. This method expands the node feature matrix of the code property graph into a multi-channel image-like matrix through the centrality analysis, and uses the graph attention network (GATv2) and the two-dimensional convolutional neural network (2D-CNN) to extract graph features. At the same time, the dual-scale one-dimensional convolutional neural network (1D-CNN) and the bidirectional long short-term memory network (BiLSTM) are used to extract sequence features. Finally, the graph features and sequence features are fused to achieve better vulnerability detection performance. After experiments on the FFMpeg+Qemu and Reveal datasets, the experimental results show that MCGCBVul outperforms the six comparison models in this paper in multiple indicators such as F1 score and accuracy, with accuracy rates of 63.952% and 92.007% respectively. In addition, this paper further proves the effectiveness of the improvement of each module of the model through ablation experiments.

Key words: Vulnerability detection; Graph neural networks; Feature fusion; Centrality analysis; Deep learning

(责任编辑: 王晓丽)